



Metoda Backtracking

Generarea elementelor combinatoriale · Backtracking în plan

Andrei Mihai Ciobanu

Algolymp Summer School

Metoda Backtracking

Elemente combinatoriale

Backtracking în plan

Alte aplicații

Recapitulare

Metoda Backtracking

Ce este backtracking?

Definiție

Backtracking este o metodă care construiește incremental o soluție, luând câte o decizie la fiecare pas — ducând astfel la parcurgerea **tuturor** soluțiilor posibile.

Tipuri de probleme:

- **Enumerare** — generează **toate** soluțiile (permutări, combinaări etc.)
- **Decizie** — există o soluție? (da / nu)
- **Optimizare** — găsește soluția **cea mai bună**

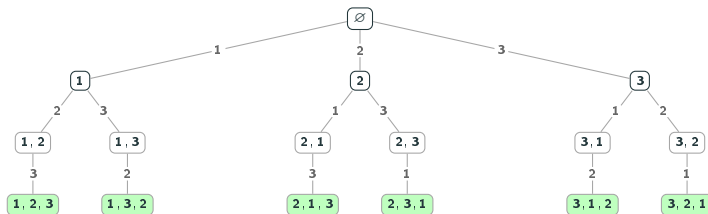
Idee centrală: construim un vector soluție $x[1], x[2], \dots, x[n]$ element cu element. La fiecare pas k :

1. Încercăm pe rând **toate valorile posibile** pentru $x[k]$
2. Pentru fiecare valoare: continuăm recursiv la pasul $k + 1$
3. Când terminăm toate valorile: **revenim** la pasul $k - 1$ (*backtrack*)

Complexitate: exponențială — în absența oricărei optimizări.

Arborele de decizie — permutări ale $\{1, 2, 3\}$

Fiecare nod = starea vectorului soluție. Ramura = valoarea aleasă la pasul curent. Frunzele verzi = soluții complete.



Arborele se explorează **în adâncime** (Depth First Search). Pentru $n = 3$: $3! = 6$ frunze = 6 permutări distincte.

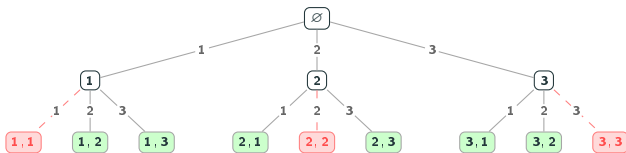
Pruning — tăierea ramurilor invalide

Definiție

Pruning înseamnă a abandona o ramură a arborelui de decizie **imediat** ce se constată că nu poate duce la o soluție validă — fără a mai explora subarboarele ei.

Exemplu: secvențe de lungime 2 din $\{1, 2, 3\}$ cu $x[1] \neq x[2]$.

Nodurile roșii sunt tăiate înainte de a mai continua recursivitatea:



Fără pruning: $3^2 = 9$ frunze. Cu pruning: **6 frunze** (3 ramuri tăiate). Cu constrângeri mai stricte (ex. N-Regine), reducerea poate fi **dramatică**.

Schelet general

```
1 int x[nmax + 1], n;
2
3 bool valid(int k) {
4     // conditia specifica problemei
5     // (x[k] compatibil cu x[1..k-1]?)
6     return true;
7 }
8 void back(int k) {
9     if (k > n) {          // solutie completa
10        // afiseaza x[1..n]
11        return;
12    }
13    for (int v = /* valori_min */; v <= /* valori_max */; v++) {
14        x[k] = v;
15        if (valid(k)) {
16            back(k + 1);
17        }
18    }
19 }
20 // Apel initial: back(1)
```

Domeniul valorilor și **condiția de soluție completă** ($k > n$) sunt specifice fiecărei probleme. `valid()` poate fi inline dacă e simplu.

Elemente combinatoriale

Factorial

$n! = 1 \cdot 2 \cdot \dots \cdot n$ — numărul de moduri de a aranja n obiecte distincte. Prin convenție: $0! = 1$.

$1! = 1$, $2! = 2$, $3! = 6$, $4! = 24$, $5! = 120$

Permutări ale $\{1, \dots, n\}$

O **permutare** este o **ordonare** a tuturor celor n elemente. Număr total:

$$P(n) = n!$$

Exemplu ($n = 3$, toate permutările mulțimii $\{1, 2, 3\}$):

$x[1]$	$x[2]$	$x[3]$
1	2	3
1	3	2
2	1	3
2	3	1
3	1	2
3	2	1

Total: $P(3) = 3! = 6$ permutări

Cheie: folosit[v] marchează elementele deja plasate $\Rightarrow$ fiecare valoare apare o singură dată.

```
1 int x[nmax + 1];
2 bool folosit[nmax + 1];
3 int n;
4
5 void back(int k) {
6     if (k > n) {
7         for (int i = 1; i <= n; i++) cout << x[i] << " ";
8         cout << "\n";
9         return;
10    }
11    for (int v = 1; v <= n; v++) {
12        if (!folosit[v]) {
13            x[k] = v;
14            folosit[v] = true;
15            back(k + 1);
16            folosit[v] = false;
17        }
18    }
19 }
20 // Apel: back(1)
```

Definiție

Un **aranjament** de p elemente din n ($p \leq n$) este o selecție **ordonată** de p elemente **distincte** din $\{1, \dots, n\}$.

$$A(n, p) = \frac{n!}{(n-p)!} = n \cdot (n-1) \cdots (n-p+1)$$

Diferența față de permutări: alegi doar $p < n$ elemente, dar **ordinea contează**: $(1, 2) \neq (2, 1)$.

Exemplu ($n = 4$, $p = 2$, din $\{1, 2, 3, 4\}$):

$(1, 2), (1, 3), (1, 4)$

$(2, 1), (2, 3), (2, 4)$

$(3, 1), (3, 2), (3, 4)$

$(4, 1), (4, 2), (4, 3)$

Total: $A(4, 2) = 4 \cdot 3 = \frac{4!}{2!} = 12$
aranjamente

Identic cu permutările — singura diferență: oprire la $k > p$ (am ales p elemente) în loc de $k > n$.

```
1 int x[nmax + 1];
2 bool folosit[nmax + 1];
3 int n, p;
4
5 void back(int k) {
6     if (k > p) {          // am ales p elemente
7         for (int i = 1; i <= p; i++) cout << x[i] << " ";
8         cout << "\n";
9         return;
10    }
11    for (int v = 1; v <= n; v++) {
12        if (!folosit[v]) {
13            x[k] = v;
14            folosit[v] = true;
15            back(k + 1);
16            folosit[v] = false;
17        }
18    }
19 }
20 // Apel: back(1)
```

Definiție

O **combinare** de p elemente din n este o selecție **neordonată** de p elemente distincte din $\{1, \dots, n\}$.

$$C(n, p) = \binom{n}{p} = \frac{n!}{p! (n-p)!} = \frac{A(n, p)}{p!}$$

Diferența față de aranjamente:

ordinea **nu** contează.

$\{1, 2\}$ și $\{2, 1\}$ sunt **aceeași** combinație.

Fiecare combinație corespunde la $p!$ aranjamente $\Rightarrow$ împărțim la $p!$.

Exemplu ($n = 4$, $p = 2$, din $\{1, 2, 3, 4\}$):

$\{1, 2\}$, $\{1, 3\}$, $\{1, 4\}$

$\{2, 3\}$, $\{2, 4\}$

$\{3, 4\}$

Total: $C(4, 2) = \frac{4!}{2! \cdot 2!} = 6$ combinații

Obs: $C(4, 2) = \frac{A(4, 2)}{2!} = \frac{12}{2} = 6 \checkmark$

Combinări — implementare

Cheie: $\text{start} = x[k-1] + 1$ forțează ordine strict crescătoare $\Rightarrow$ fără permutări duplicate. Limita $n - p + k$ evită porniri imposibile.

```
1 const int nmax = 10;
2 int x[nmax + 1];
3 int n, p;
4
5 void back(int k) {
6     if (k > p) {
7         for (int i = 1; i <= p; i++) {
8             cout << x[i] << " ";
9         }
10        cout << "\n";
11        return;
12    }
13    int start = (k == 1) ? 1 : x[k - 1] + 1;
14    for (int v = start; v <= n - p + k; v++) {
15        x[k] = v;
16        back(k + 1);
17    }
18 }
19 // Apel: back(1)
```

Definiție

O **submulțime** a lui $\{1, \dots, n\}$ este orice selecție de elemente (inclusiv mulțimea vidă $\emptyset$ și întreaga mulțime).

2^n submulțimi în total

De ce 2^n ? Fiecare din cele n elemente poate fi **inclus sau exclus** independent:

$$\underbrace{2 \times 2 \times \dots \times 2}_{n \text{ elemente}} = 2^n$$

Spre deosebire de combinaări, nu e fixat numărul de elemente alese.

Exemplu ($n = 3$, mulțimea $\{1, 2, 3\}$):

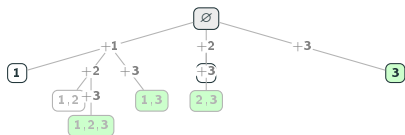
Mască	Submulțime
000	$\emptyset$
001	$\{1\}$
010	$\{2\}$
011	$\{1, 2\}$
100	$\{3\}$
101	$\{1, 3\}$
110	$\{2, 3\}$
111	$\{1, 2, 3\}$

Total: $2^3 = 8$ submulțimi

Submulțimi — BT: alegere element următor

Afișăm submulțimea curentă la fiecare apel, apoi **extindem** cu elemente mai mari decât ultimul ales.

```
1 int x[nmax + 1], len, n;
2
3 void back(int last) {
4     for (int i = 1; i <= len; i++) cout << x[
5         i] << " ";
6     cout << "\n";
7     for (int v = last + 1; v <= n; v++) {
8         x[++len] = v;
9         back(v);
10        len--;
11    }
12 }
13 // Apel: back(0)
```



2^n apeluri — câte unul per submulțime.

Submulțimi — implementare (măști de biți)

Fiecare întreg $\text{mask} \in [0, 2^n)$ codifică o submulțime: bitul $i = 1$ înseamnă că elementul $i + 1$ este inclus.

```
1 int a[nmax + 1], n;
2
3 for (int mask = 0; mask < (1 << n); mask++) {
4     for (int i = 0; i < n; i++) {
5         if (mask & (1 << i)) {
6             cout << a[i + 1] << " ";
7         }
8     }
9     cout << "\n";
10 }
```

Eficient pentru $n \leq 20$ — $2^{20} \approx 10^6$ iterații. **Bonus — combinări cu bitmask:** selectăm doar măștile cu exact p biți setați (folosind `__builtin_popcount(mask) == p`). Mai ineficient decât BT clasic, dar util când avem deja bitmask-ul submulțimii.

Definiție

Produsul cartezian $A_1 \times A_2 \times \cdots \times A_n$ este mulțimea tuturor secvențelor $(x_1, \dots, x_n)$ cu $x_k \in A_k$ pentru fiecare k .

$$|A_1 \times \cdots \times A_n| = \prod_{k=1}^n |A_k|$$

Diferența față de permutări: valorile nu trebuie să fie distincte; fiecare poziție are **propriul domeniu** de valori.

Dacă $|A_k| = m$ pentru orice k : total m^n secvențe.

Exemplu: $A_1 = \{1, 2\}$, $A_2 = \{1, 2, 3\}$

Toate perechile (x_1, x_2) :

x_1	x_2
1	1
1	2
1	3
2	1
2	2
2	3

Total: $2 \times 3 = 6$ secvențe
(obs: $(1, 1)$ și $(2, 2)$ sunt valide — repetiții permise!)

Produs cartezian — implementare

Cheie: fără folosit[] — fiecare poziție k parcurge independent valorile $1 \dots \text{lim}[k]$ (repetiții permise).

```
1 const int nmax = 10;
2 int x[nmax + 1];
3 int lim[nmax + 1];
4 int n;
5
6 void back(int k) {
7     if (k > n) {
8         for (int i = 1; i <= n; i++) {
9             cout << x[i] << " ";
10        }
11        cout << "\n";
12        return;
13    }
14    for (int v = 1; v <= lim[k]; v++) {
15        x[k] = v;
16        back(k + 1);
17    }
18 }
19 // Apel: back(1)
```

Backtracking în plan

Problema N Reginelor — formulare

Problemă

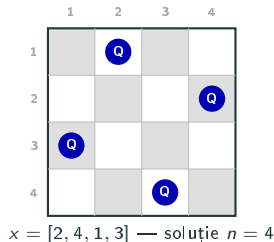
Plasează n regine pe o tablă de șah $n \times n$ astfel încât nicio două regine să nu se atace reciproc (pe linie, coloană sau diagonală).

Modelare: $x[i]$ = coloana reginei de pe rândul i . Deoarece punem exact o regină per rând, conflictele pe linie sunt excluse automat. Rămân de verificat:

- **Coloană:** $x[i] \neq x[j]$ pentru $i \neq j$
- **Diagonală:** $|x[i] - x[j]| \neq |i - j|$ pentru $i \neq j$

Complexitate: exponențială, dar pruning-ul reduce semnificativ spațiul de căutare.

Pentru $n = 8$: **92 soluții**.



N Regine — valid()

Verificăm noua regină față de toate cele plasate anterior pe rândurile $1 \dots k-1$.

```
1 const int nmax = 20;
2 int x[nmax + 1];
3 int n;
4
5 bool valid(int k) {
6     for (int i = 1; i < k; i++) {
7         if (x[i] == x[k]) {
8             return false; // aceeași coloana
9         }
10        if (abs(x[i] - x[k]) == abs(i - k)) {
11            return false; // aceeași diagonală
12        }
13    }
14    return true;
15 }
```

Optimizare $O(1)$: în loc de bucla din `valid()`, menține tablouri `col[c]`, `diag1[r-c+n]`, `diag2[r+c]` marcând coloanele/diagonalele ocupate.

N Regime — back()

```
1 void back(int k) {
2     if (k > n) {
3         for (int i = 1; i <= n; i++) { cout << x[i] << " "; }
4         cout << "\n";
5         return;
6     }
7     for (int col = 1; col <= n; col++) {
8         x[k] = col;
9         if (valid(k)) {
10             back(k + 1);
11         }
12     }
13 }
14 // Apel: back(1)
```

Pruning: `valid()` taie ramura imediat — pentru $n = 8$ se vizitează doar ~ 15000 noduri din $8^8 \approx 16 \times 10^6$ posibile. Total **92 soluții** pentru $n = 8$.

Traseul Calului (*Knight's Tour*)

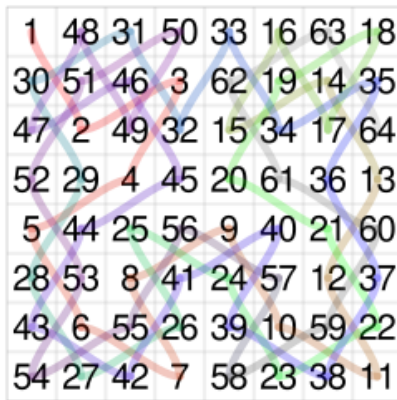
Problemă

Plasează un cal de șah pe o tablă $n \times n$ astfel încât să viziteze fiecare celulă **exact o dată** (lanț hamiltonian).

Calul se mișcă în L: $(\pm 1, \pm 2)$ sau $(\pm 2, \pm 1)$ — 8 mișcări posibile per celulă.

Algoritmul Warnsdorff (1823) — o euristică

Alege la fiecare pas mișcarea care duce în celula cu **cele mai puține ieșiri nevizitate**.
BT pur: $O(8^{n^2})$. Cu Warnsdorff: $O(n^2)$ în practică.



Traseu complet pe tablă 8×8 (numerele indică ordinea mișcărilor)

Traseul Calului — implementare BT

```
1  const int nmax = 8;
2  int tabla[nmax + 1][nmax + 1];
3  int dx[] = {-2,-2,-1,-1, 1, 1, 2, 2};
4  int dy[] = {-1, 1,-2, 2,-2, 2,-1, 1};
5  int n;
6  bool valid(int x, int y) {
7      return x >= 1 && x <= n && y >= 1 && y <= n
8          && tabla[x][y] == 0;
9  }
10 void back(int x, int y, int pas) {
11     tabla[x][y] = pas;
12     if (pas == n * n) {
13         // afiseaza tabla[][]
14         tabla[x][y] = 0; return;
15     }
16     for (int d = 0; d < 8; d++) {
17         int nx = x + dx[d], ny = y + dy[d];
18         if (valid(nx, ny)) { back(nx, ny, pas + 1); }
19     }
20     tabla[x][y] = 0; // backtrack
21 }
22 // Apel: back(1, 1, 1)
```

Alte aplicații

Backtracking — când e optim și când nu

- **Permutări / Aranjamente / Combinări / Submulțimi**
→ BT **optim** (trebuie generate toate soluțiile)
- **N-Regine, Sudoku, Colorarea grafurilor**
→ BT cu pruning **standard**; soluție exponențială inevitabilă
- **Rucsac 0/1** — DP $O(nW)$ când W mic; **BT + Branch & Bound** când W enorm
(100p, ≤ 1 ms/test)
- **Problema spectacolelor**
BT: $O(2^n)$ mai bine: Greedy $O(n \log n)$
- **Traseul Calului** — BT pur: exponențial mai bine: Warnsdorff $O(n^2)$
- **Lanț / ciclu hamiltonian, SAT, Colorare optimală**
→ Probleme **NP-complete**: BT e abordarea clasică

Regulă practică: dacă $n \leq 20$ –25, BT e viabil. Dacă n e mare, caută structură exploatabilă (DP, Greedy, B&B ...).

Problema

n obiecte cu greutate w_i , profituri p_i , rucsac de capacitate W . Maximizăm profitul fără a depăși W .

De ce nu DP? $O(nW)$ — imposibil la $W \leq 10^9$.

BT + B&B: 100p pe infoarena,
 ≤ 1 ms/test.

Cele 4 optimizări:

1. **Sortare** după p_i/w_i descrescător
2. **Bound optimist**: rucsac fracționar ca upper bound
3. **Init greedy**: max_profit bun $\Rightarrow$ pruning agresiv
4. **Take first**: luăm înainte de a renunța la obiect

Dacă

$profit + bestSuffix(i, W - weight) \leq max_profit$:

$\rightarrow$ tăiem ramura (pruning).

Rucsac 0/1 — implementare B&B

```
1 // a[i] = {w, p}, sortat dupa p/w desc; max_profit initializat greedy
2 long long weight = 0, profit = 0, max_profit;
3 long double bestSuffix(int i, long long rem) {
4     long double best = 0;
5     for (int j = i; j <= n && rem > 0; ++j) {
6         long long aux = min(rem, a[j].first);
7         best += 1.0L * a[j].second * aux / a[j].first;
8         rem -= aux;
9     }
10    return best;
11 }
12 void backtrack(int i) {
13     if (i > n) { max_profit = max(max_profit, profit); return; }
14     if (profit + bestSuffix(i, W - weight) <= max_profit) return;
15     weight += a[i].first; profit += a[i].second;
16     if (weight <= W) backtrack(i + 1);
17     weight -= a[i].first; profit -= a[i].second;
18     backtrack(i + 1);
19 }
```

Idee: pentru permutarea curentă, găsim *succesoarea* în ordine lexicografică, repetând până epuizăm toate $n!$ permutări.

Algoritmul (3 pași)

1. Găsim cel mai din dreapta i cu $x[i] < x[i + 1]$
(prefixul descrescător de la final)
2. Găsim cel mai mic $x[j] > x[i]$ cu $j > i$
și îl schimbăm cu $x[i]$
3. Inversăm sufixul $x[i + 1 .. n]$

Exemplu $\{1, 3, 2\} \rightarrow \{2, 1, 3\}$:

$$x[1]=1 < x[2]=3 \Rightarrow i = 1$$

Cel mai mic $x[j] > 1$ cu $j > 1$:

$$x[2]=2$$

(după inversarea sufixului

$$3, 2 \rightarrow 2, 3)$$

$$\text{Swap } x[1] \leftrightarrow x[2]: \{2, 1, 3\}$$

Inversare sufix: $\{2, 1, 3\}$

Complexitate: $O(n)$ /permutare;
 $O(n \cdot n!)$ total.

Identic ca backtracking, dar fără
stivă recursivă.

next_permutation — implementare

```
1 bool nextPermutation(int x[], int n) {
2     int i = n - 1;
3     while (i > 0 && x[i] >= x[i + 1]) i--;
4     if (i == 0) return false; // ultima permutare
5     int j = n;
6     while (x[j] <= x[i]) j--;
7     swap(x[i], x[j]);
8     // inversam sufixul x[i+1..n]
9     int l = i + 1, r = n;
10    while (l < r) { swap(x[l], x[r]); l++; r--; }
11    return true;
12 }
13
14 // Generare toate permutarile lui {1..n}:
15 // initializare: x[i] = i (permutarea 1,2,...,n)
16 do {
17     for (int i = 1; i <= n; i++) cout << x[i] << " ";
18     cout << "\n";
19 } while (nextPermutation(x, n));
20 // SAU direct: std::next_permutation(x + 1, x + n + 1)
```


Recapitulare

Termeni importanți

- **Backtracking** — metodă de construire incrementală a unei soluții prin explorarea **tuturor** posibilităților (DFS în arborele de decizie). La fiecare pas încearcă toate valorile, după care **revine** (*backtrack*) la pasul anterior.
- **Arbore de decizie** — structură în care fiecare nod reprezintă o stare parțială a soluției, fiecare ramură o decizie, iar frunzele soluțiile complete (sau ramuri tăiate).
- **Pruning** (tăierea ramurilor) — abandonarea unui subarbore **imediat ce** se constată că nu poate duce la o soluție validă, fără a-l mai explora. Reduce exponențial spațiul de căutare la probleme cu constrângeri stricte (ex. N-Regine, Sudoku).
- **Euristică** — regulă de decizie **aproximativă** care ghidează căutarea spre soluții bune fără a garanta optimalul. Exemplu: algoritmul Warnsdorff pentru traseul calului (alege celula cu cele mai puține ieșiri libere) — funcționează excelent în practică, dar nu e demonstrat matematic pentru orice n .

Probleme educationale

1. **All Permutations of 1 to N** (AlgoCode #1704)
2. **Cartesian Product Generation** (AlgoCode #1705)
3. **N Queens Board** (AlgoCode #1364)
4. **Arrangements of k Elements** (AlgoCode #1373)
5. **Subsets of k Elements** (AlgoCode #1374)
6. **Nonempty Subsets** (AlgoCode #1375)
7. **Distinct Word Anagrams** (AlgoCode #1376)
8. **Balanced Bracket Strings** (AlgoCode #1412)

Probleme de antrenament

1. **Anagrams with Fixed Vowels** (AlgoCode #1379)
2. **Binary Sequences with Same Digit Counts** (AlgoCode #1383)
3. **Constrained Digit Generation** (AlgoCode #1386)
4. **Increasing Sum Decompositions** (AlgoCode #1403)
5. **Signed Squares Equal N** (AlgoCode #1404)
6. **Bounded Banknote Payment** (AlgoCode #1405)
7. **Cube Towers** (AlgoCode #1406)
8. **Distinct Sum Decompositions** (AlgoCode #1411)
9. **Fazan Word Chains** (AlgoCode #1414)
10. **Permutations with Min and Max Fixed** (AlgoCode #1703)
11. **Blocked Queens** (AlgoCode #4395)
12. **Digit Difference Numbers** (AlgoCode #1370)
13. **Special Increasing Sequences** (AlgoCode #1706)
14. **Reciprocal Sum Sequences** (AlgoCode #1707)

- **pbinfo.ro** — Capitolul “Metoda Backtracking”
Teorie + pseudocod + probleme clasice (permutări, regine, labirint)
- **pbinfo.ro** — “Generarea permutărilor / aranjamentelor / combinărilor / submulțimilor”
Cod complet pentru fiecare tip de generare
- **GeeksforGeeks** — “Introduction to Backtracking”
Arbore de stări, pruning, N-Queens, Sudoku cu explicații vizuale
- **HackerEarth** — “Recursion and Backtracking”
Recursivitate ca mecanism, backtracking ca strategie, exemplu labirint
- **Wikipedia** — “Eight queens puzzle”
92 soluții pentru $n = 8$, variante, complexitate
- **Wikipedia** — “Knight’s tour”
Algoritmul Warnsdorff, pătrate magice cu traseu de cal